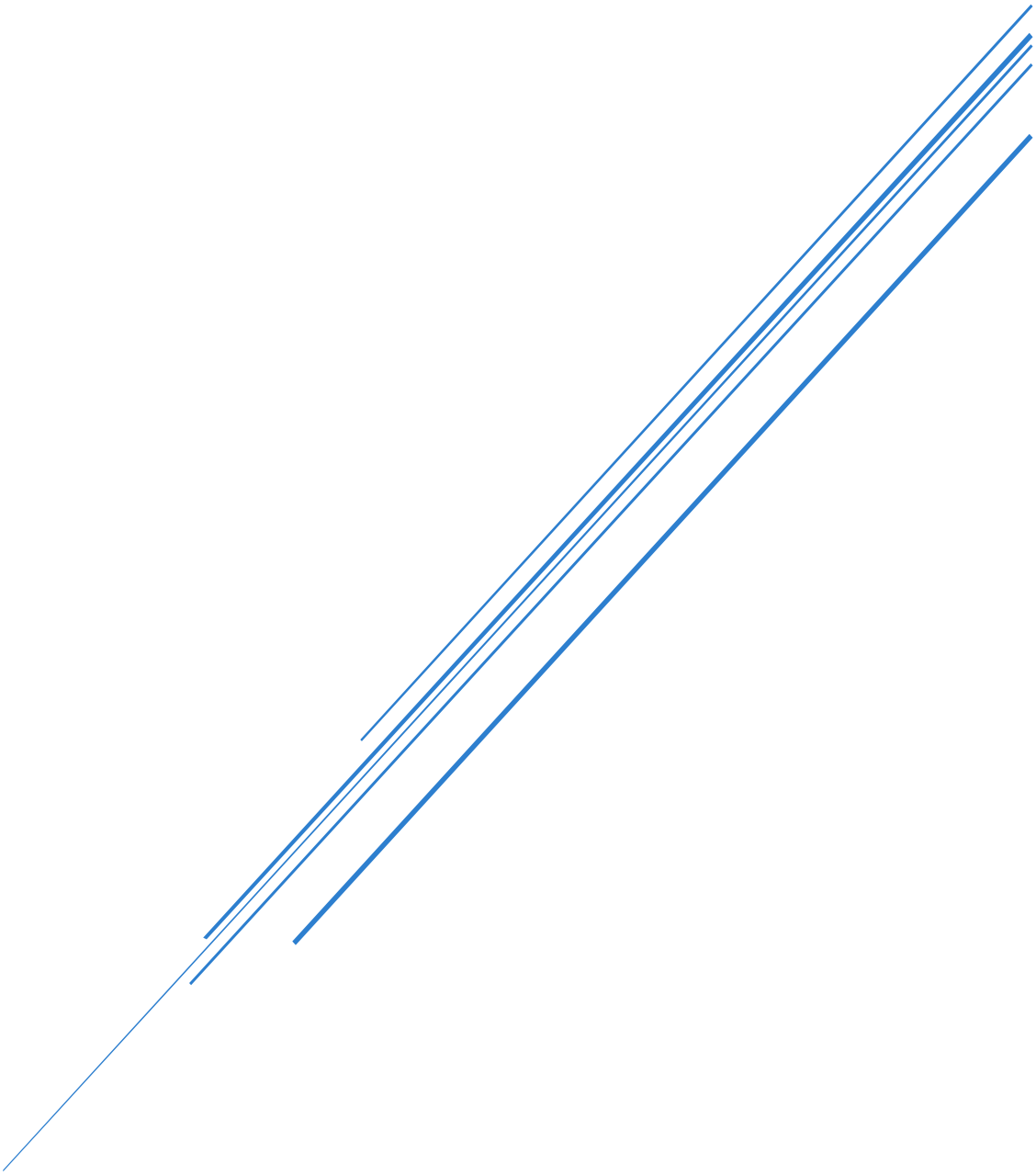
TORQ

Requirement Specification for a web app called TORQ.



Contents

Application Purpose 2
Target Audience 3
User Profiles 3
Timeline ..... **Error! Bookmark not defined.**
Design Implications 6
  Colour Palette and Branding 6
  Dashboard Layout 6
Development Lifecycle 7
Requirements 8
  Functional Requirements 8
  Non-functional Requirements 8
System Requirements 9
  Hardware Requirements 9
  Foundations (language, DBMS) 9
  Development Requirements 9
API Ninjas Motorcycle API 10
  What It Does 10
  Role in the App 10
  Implementation 10
Database Design 11
  Entity Relationship Diagram 11
  Independent Reference Tables 11
  Design Notes 11
Weather API 12
  What It Does 12
  Role in the App 12
  Implementation 12
Project Plan 13
Navigation Structure 13
Constraints 14
What I will learn 14

Application Purpose

TORQ is a web-based application design to support the motorcycle community, new and old riders, in bike maintenance, safety improvements, and to engage the rider community within Glasgow. The application aims to allow users to:

- Track motorcycle maintenance tasks with dates. Users will input their current bike knowledge, and an algorithm will determine an estimate for next maintenance checks. For example, oil in 3 months, tyre pressure weekly etc.
- Receive reminders on the dashboard for upcoming servicing.
- Users can **optionally** input their MOT, Tax and Insurance renewal dates, and it will display on their user profile.
- A static pre-ride safety checklist (useful for new riders) detailing all the quick checks they should do before they ride.
- Display local events with days, times, info, and pictures. (these can be filled with placeholder info. Must include event Name, Date, start and end time, description, and IMG for posters)
- Display hot spots, places where bikes regularly accumulate, nice views, ride routes etc.
- Display guides on common bike issues such as how to change oil, how to check brakes, etc.

The system will be data-driven, using user input to dynamically generate personalised outputs.

The goal of TORQ is to improve rider safety and awareness of their bike. Especially for new and younger riders who are just joining the world of motorcycling and may not quite understand their bike. It will also be useful to the older generation of riders who find themselves picking up bad habits or simply forgetting the basics of bike maintenance. It can be a very useful all-in-one tool for people to keep themselves safe and engage within the community via meetups, events, rides.

Target Audience

The application is aimed at a wide range of motorcycle riders including:

- New riders (16+) who are inexperienced in maintenance.
- Non-riders (possibly under 16) who want to know how to obtain a motorcycle license.
- Intermediate riders who require reminders and organisation tools
- Experienced riders who wish to track service history and engage with the local community
- Riders who don't need to track maintenance but want to browse ride outs.

The system will be designed to be accessible to users of varying technical ability through a clean, minimal interface. The web-app will be developed as accessible as possible, adhering to WCAG guidelines.

User Profiles

User Profile 1	
Name	Jamie
Age	19
License Type/Experience	Recently passed CBT and bought 1st 125cc bike
Needs:	Simple pre ride checklist to ingrain the information until it becomes second nature. Has never owned a bike or car before so doesn't know what the necessary steps are to look after it. Would like to see features that are focused around their safety and making them a better rider.
User Profile 2	
Name	Callum
Age	32
License Type/Experience	A license. Owns multiple bikes. Does own servicing.
Needs:	Needs to track the service history and log maintenance. Is always looking for a new meet up spot, goes to as many events as possible, wants to see them all in the place instead of searching for them.
User Profile 3	
Name	Sarah

Age	27
License Type/Experience	A license. Owned her bike for a few years. Not consistent with upkeep.
Needs:	Ease of use reminders. A simple interactive interface
User Profile 4	
Name	Eddie
Age	65
License Type/Experience	A license. Hasn't owned a bike in 15 years but wants back on the road. Looked after own bikes basic maintenance.
Needs:	Not tech savvy so needs simplistic UI. All his old hotspots are quiet and wants new places to explore. Doesn't know anyone with a bike so would explore the community aspect. Has forgotten a lot of basic maintenance so links to guides and a logger would be useful but not essential.

Project Plan

Week 1 – 2	• Write up proposal. • Research the needs within the bike community (done within a whatsapp group chat of rider friends) • Research how to implement certain features. • Create initial wireframes. • Requirement Spec
Week 3 – 4	• Design the database and how it all links together. • Set up the backend (Node.js and express) • Create the database on MySQL and connect it to the backend just to make sure it is working. • Potentially create the info and events pages as they only generate dynamically and do not update in real time like the user dashboard.
Week 5 – 6	• Finalise the user authentication system. • Ensure it is secure. • Begin creating the framing for the frontend with React. • Make it look pretty. • Begin work on maintenance tracking (in the end, this was scoped down to editable service snapshots rather than a full calculation engine – see Requirements).
Week 7 – 8	• Create the user dashboard and populate dynamically. • Implement features.
Week 9	Continue development.
Week 10	• Testing and debugging. • Find peers in the community to test. • Create a survey for users to answer. • Collect and collate findings. • Fix any bugs after testing and retest. • Begin evaluation
Week 11	Finish evaluation.

Design Implications

The design of TORQ must consider the needs of a wide and varied audience, ranged ages 16-70.

- The interface will follow a **minimalist, modern design approach** to ensure usability for non-technical users
- A **default dark mode theme** will be implemented to reduce eye strain and provide a modern appearance
- The dashboard will use a **card-based layout** to present key information clearly
- Buttons and interactive elements will be **large, bold and clearly labelled** to improve accessibility
- Navigation will be simple, familiar, and responsive to different viewports.
- The application will be fully **responsive**, ensuring usability on mobile, tablet, and desktop. It will be developed with a mobile first approach.
- High contrast colours will be used to improve readability
- The system will prioritise **ease of use over complexity**, particularly for new riders

Colour Palette and Branding

The brand mark is a white helmet icon paired with the wordmark TORQ. The dark mode theme uses a near-black dark blue rather than pure black or grey, to feel more branded and less generic:

- Page background: near-black dark blue (#0A0E1A)
- Card surface: dark navy (#131B2E), sitting one step lighter than the page background so cards are visible without borders or shadows
- Primary text: near-white (#F5F7FA); secondary and supporting text: muted blue-grey (#8B95AC)
- Accent colour: orange (#FF7A33), used **only** where it carries meaning – the active navigation tab, urgent maintenance alerts, click and hover states, progress indicators, and card border accents. It is never used decoratively, so it keeps its visual weight as a signal for "this needs attention" or "this is active".

Dashboard Layout

The original plan described a separate Home dashboard and Profile screen, with a two-column "Your bike"/"Community" split and a desktop sidebar. During development, having both a "Profile" and a "My Dashboard" entry in the navigation proved confusing to test with, so the two were deliberately merged into a single signed-in page, described here as it was actually built.

The merged page is laid out top to bottom as:

- **Account header** – avatar, display name and email, with an edit action
- **My bikes** – stackable bike cards (FR24), each expandable to show key dates, mileage and full specs (FR10, FR18), plus an "Add another bike" action
- **Ride conditions and featured event** – today's weather (FR15) and the single soonest upcoming event, side by side on wider screens and stacked on mobile
- **Pre-ride checklist** – the static safety checklist (FR6)

The whole page is a single stacked column on mobile. On wider screens, only the ride conditions and featured event row splits into two side-by-side cards; everything else remains a single column, so there is no separate desktop-only sidebar.

Development Lifecycle

The project will follow an Agile development approach, allowing for iterative development and improvement. I believe this will benefit me and my learning the best, as it allows me to be ambitious but also keep within my abilities.

Development will be broken into stages (planning, development, testing, evaluation). Features can be created modularly and implemented in increments. Testing can occur throughout development, allowing me to focus on tasks and at the end before submission, fix any bugs fine tune the design etc. Feedback will be gathered throughout the project and importantly, at the end, and will be used to refine the overall design and functionality of the app.

Requirements

Functional Requirements

FR1	Allow users to log in securely (like hashing password)
FR2	Allow users to create and manage a bike profile
FR3	Store the bike's most recent service date and mileage
FR5	Display maintenance reminders on dashboard
FR6	Provide a pre-ride safety checklist
FR8	Display local events and hotspots dynamically
FR9	Display guides on maintenance, possibly hyperlinking to other sites?
FR10	Display important vehicle dates such as MOT, insurance
FR11	Store and retrieve user specific user data from a database
FR12	Personalised user dashboard
FR15	Display today's weather conditions on the dashboard, to support pre-ride safety decisions
FR16	Provide search and filter within Guides, so users can find content directly rather than only browsing
FR17	Show a clear empty state (e.g. "Add your bike") on the dashboard and bike snapshot when a new user has not yet created a bike profile
FR18	Display a "last synced" timestamp on cached bike specification data, so users know how current the information is
FR20	Show clear loading and error states for any dynamically fetched content (events, hotspots, bike specification data)
FR21	Allow new users to register an account
FR22	Allow users to securely log out, clearing session data
FR24	Allow users to add and manage multiple bikes under one account

Non-functional Requirements

NFR1	Be responsive across mobile, tablet, desktop devices
NFR2	Load within 2 seconds under normal conditions
NFR3	Consistent interface, dark mode for an easy look on the eyes
NFR4	Securely store passwords using hashing
NFR5	User-friendly and easy to navigate
NFR6	Follow WCAG accessibility standards
NFR7	Be accessible to a variety of users
NFR8	Maintain data integrity, privacy and consistency
NFR9	Smooth interactions and transitions
NFR10	Show loading states and/or error messages so the screen isn't just blank
NFR11	Cache external API responses where practical to reduce repeat calls and stay within free-tier usage limits (Storing postcode lat/long entries, storing bike data after returning data from Motorcycles API)

System Requirements

Hardware Requirements

- Computer, laptop or mobile
- Internet connection
- Peripherals

Foundations (language, DBMS)

The application will be developed using React, Node.js With Express, MySQL and Tailwind for styling. All of these work together well, are scalable and flexible, and relevant to the current industry.

Development Requirements

- Node.js (Backend)
- Express.js (API handler)
- React (Frontend)
- MySQL (Database)
- XAMPP (Local server)
- Visual Studio Code (IDE)
- Git for version control
- Trello (Project management)
- Figma and Figjam (wireframing)
- Open-Meteo (weather data, FR15)
- postcodes.io (postcode to coordinates lookup, supports the weather feature)

API Ninjas Motorcycle API

What It Does

API Ninjas provides a Motorcycles API containing specification data on tens of thousands of motorcycle models from hundreds of manufacturers. It has three endpoints, though only one is available on the free tier used by this project:

- /v1/motorcyclemakes and /v1/motorcyclemodels would return manufacturer and model lists, but are restricted to paid Business/Professional API Ninjas plans, so are not used by this project
- /v1/motorcycles?make=X&model=Y&year=Z is available on the free tier, returns full technical specifications for a specific make, model and year, and supports partial matching on the model parameter, which this project relies on for live search suggestions (the user will type their model and the search debounces for results)

The API is used under its free tier for this site.

Role in the App

The API powers the “My Bike” profile creation flow (FR2). A user selects their bike’s make from a fixed list, types their model with suggested results, and picks a year, rather than typing every field as free text (there is too much to go wrong with that). The retrieved specification data is also used to populate the bike profile’s specs section (FR12), giving each user a personalised data driven overview of their motorcycle.

Implementation

- The Make field is a short static list of common manufacturers held in code, since the live makes endpoint is not available on the free tier of this API.
- The Model field is a text input. Once a make is chosen and the user has typed a couple of characters, the app calls /v1/motorcycles?make=X&model=Y (debounced) to show live matching suggestions, using the endpoint’s partial match support; if nothing matches, the user’s typed text is kept as free text rather than blocking them.
- The results of the models all have a year attached.
- On submission, the backend calls /v1/motorcycles once with the chosen make, model and year, and stores the returned JSON specification data in the bikes table in MySQL, linked to the user’s account.
- Caching the response at creation time, rather than calling the API on every dashboard load, keeps the dashboard within the 2-second load target (NFR2) and keeps the app functional even if the third-party API is temporarily unavailable.

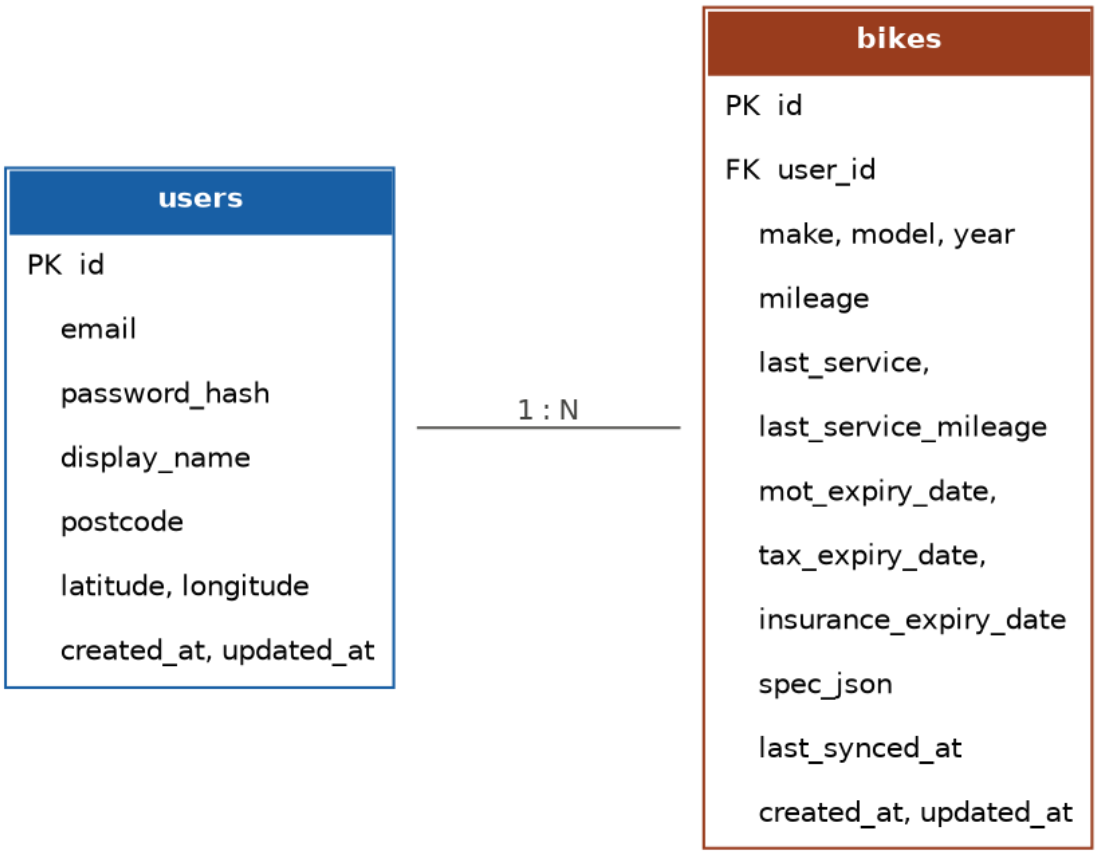
The API doesn’t have bike images so I will use a Tabler Icon instead.

Database Design

The database will be implemented in MySQL. The design below covers the core entities needed for FR2–FR5, FR10–FR12 and FR24. Currently only supports one bike entry at a time.

Entity Relationship Diagram

The three core relational tables and their keys are shown below. Independent reference tables are described separately, as they do not hold foreign keys to a user.



Independent Reference Tables

These tables hold data that is not owned by an individual user, so they carry no foreign key back to the users table.

- **events** (id PK, name, description, event_date, start_time, end_time, location, image_url – nullable, created_at, updated_at)
- **hotspots** (id PK, name, description, location, image_url – nullable, created_at, updated_at)

Design Notes

- bikes.spec_json stores the cached API Ninjas response (see API Ninjas Motorcycle API section), so a bike's specification data survives even if the third-party API is temporarily unavailable.
- Rather than a full maintenance_records log, each bike stores a single last_service and last_service_mileage snapshot, updated directly by the user (FR3). A proper service

history per bike, matching the original design, is a future addition rather than a change to the current site. I wanted this but I think it will be out of current scope.

- mot_expiry_date, tax_expiry_date and insurance_expiry_date are nullable, since these fields are optional per the Application Purpose section.
- Passwords are never stored in plain text; users.password_hash holds a bcrypt hash only, per NFR4.

Weather API

What It Does

Open-Meteo is used to provide the dashboard weather data. It is free for non commercial use, requires no API key or account, and returns current conditions plus an hourly forecast as plain JSON over a simple HTTP GET request. Relevant fields are temperature, precipitation probability, wind speed and gusts, and a weather code describing conditions such as fog, rain or thunderstorms. Not needing an API key removes a credential to manage and secure, which fits well alongside the API Ninjas integration.

Role in the App

The dashboard displays today's conditions for the user's area like the temperature, general conditions and wind based on their saved postcode, defaulting to Glasgow if none is set (FR15) so a rider can check conditions alongside the pre-ride checklist before setting off.

Implementation

- The Express backend calls Open-Meteo's forecast endpoint with the user's stored latitude and longitude, geocoded once from their postcode via postcodes.io and saved on their user record (defaulting to Glasgow if no postcode is set), requesting current conditions.
- The postcode-to-coordinates lookup only happens once, at the point the user sets their postcode, rather than on every dashboard load. That is the caching this feature relies on (NFR11). The weather reading itself is fetched fresh from Open-Meteo on each dashboard load.
- If Open-Meteo is unavailable, the dashboard shows a clear error state rather than a blank or broken weather card, per FR20 and NFR10.

Project Plan

The project will be managed on Trello and developed over an 11 week period.

Week 1 – 2	• Write up proposal. • Research the needs within the bike community (done within a whatsapp group chat of rider friends) • Research how to implement certain features. • Create initial wireframes. • Requirement Spec
Week 3 – 4	• Design the database and how it all links together. • Set up the backend (Node.js and express) • Create the database on MySQL and connect it to the backend just to make sure it is working. • Potentially create the info and events pages as they only generate dynamically and do not update in real time like the user dashboard.
Week 5 – 6	• Finalise the user authentication system. • Ensure it is secure. • Begin creating the framing for the frontend with React. • Make it look pretty. • Begin work on maintenance tracking (in the end, this was scoped down to editable service snapshots rather than a full calculation engine – see Requirements).
Week 7 – 8	• Create the user dashboard and populate dynamically. • Implement features.
Week 9	Continue development.
Week 10	• Testing and debugging. • Find peers in the community to test. • Create a survey for users to answer. • Collect and collate findings. • Fix any bugs after testing and retest. • Begin evaluation
Week 11	Finish evaluation.

Navigation Structure

The application will include:

- Home
- Guides
- Community (events and hotspots together on one page)
- Dashboard

Navigation will be designed different depending on viewport. Mobile will feature a drop down menu via hamburger, whereas tablet and desktop will have a horizontal list in the top right corner.

Constraints

- Time limitations – I only have until May 29th to develop the site. (Now Aug 8th)
- User interactivity will be somewhat limited, being unable to post like a social media feed.
- Data is only as accurate as the users input.
- Application is likely to be focused within the one region, however in future could be expanded.
- Need to learn more about APIs. I have very little knowledge of them but thankfully the ones I want to use come with a lot of guide / documentation so should be an easy enough tutorial.
- Heavily data driven, although I have worked with Node, MySQL and express before, it was tough. I risk spending more time debugging things I am unfamiliar with.
- I have very limited experience with React as a frontend framework, so will have to learn as I go.
- I don't have the sharpest skills when It comes to design, creating logos or graphics.
- User data must be protected, but full company level security is out of project scope

What I will learn

The development of TORQ will require a lot of learning, improving and honing in skills. These include:

- React framework and its component based architecture.
- Uploading and pulling data back down from a database. Although I have done this before, I have never had to do it with so so much data. The tables are quite in depth and there's a lot to move around.
- Chaining an API with another. I expect the postcode -> lat/long will be a pain but with guidance it should be alright.
- JWT Auth tokens
- Major deep dive into tailwind. In the past I have used templates and imported them into PHP but this will be my first time properly learning Tailwind. Before when doing plain CSS, HTML and JS I had a similar approach to class names (creating class variables and using them, rather than taking a container class and styling it individually). I think this will translate to tailwind and it will become a case of learning tailwinds names.
- React Router